PHP

CRÉATION DE PAGES WEB DYNAMIQUES CÔTÉ SERVEUR

Dr Pascal FAYE

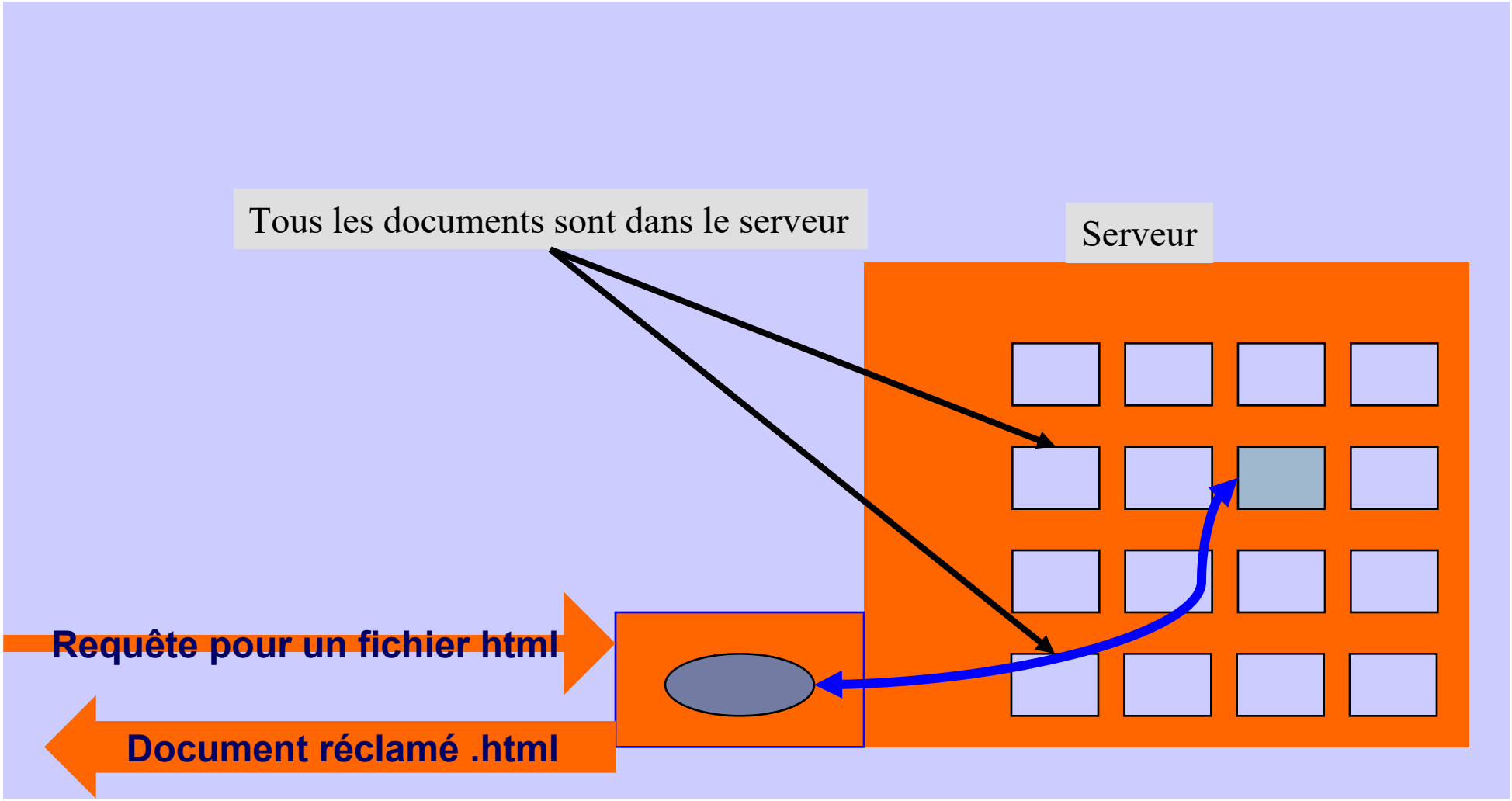
Introduction(1)

- Internet et les pages web
  - HTML (HyperText Markup Language) : conception de pages destinées à être publiées sur Internet. Une page html contient le texte à afficher et des instructions de mise en page.
  - HTML est un langage de description de page et non pas un langage de programmation.
    - pas d'instructions de calcul ou pour faire des traitements suivant des conditions.
  - Des sites de plus en plus riches en informations :
    - Nécessité croissante d'améliorer le contenu de sites.
    - Mises à jour manuelles trop complexes .
    - Automatiser les mises à jour ?

Introduction(2)

- Pages web statiques : fonctionnement
  - Leurs contenus ne changent ni en fonction du demandeur ni en fonction d'autres paramètres éventuellement inclus dans la requête adressée au serveur. Toujours le même résultat.
    - Rôle du serveur :
      - localiser le fichier correspondant au document demandé.
      - répondre au navigateur en lui envoyant le contenu de ce fichier.
- Pages web statiques : limites
  - Besoin de réponses spécifiques, d'où:
    - le passage des pages statiques aux pages dynamiques

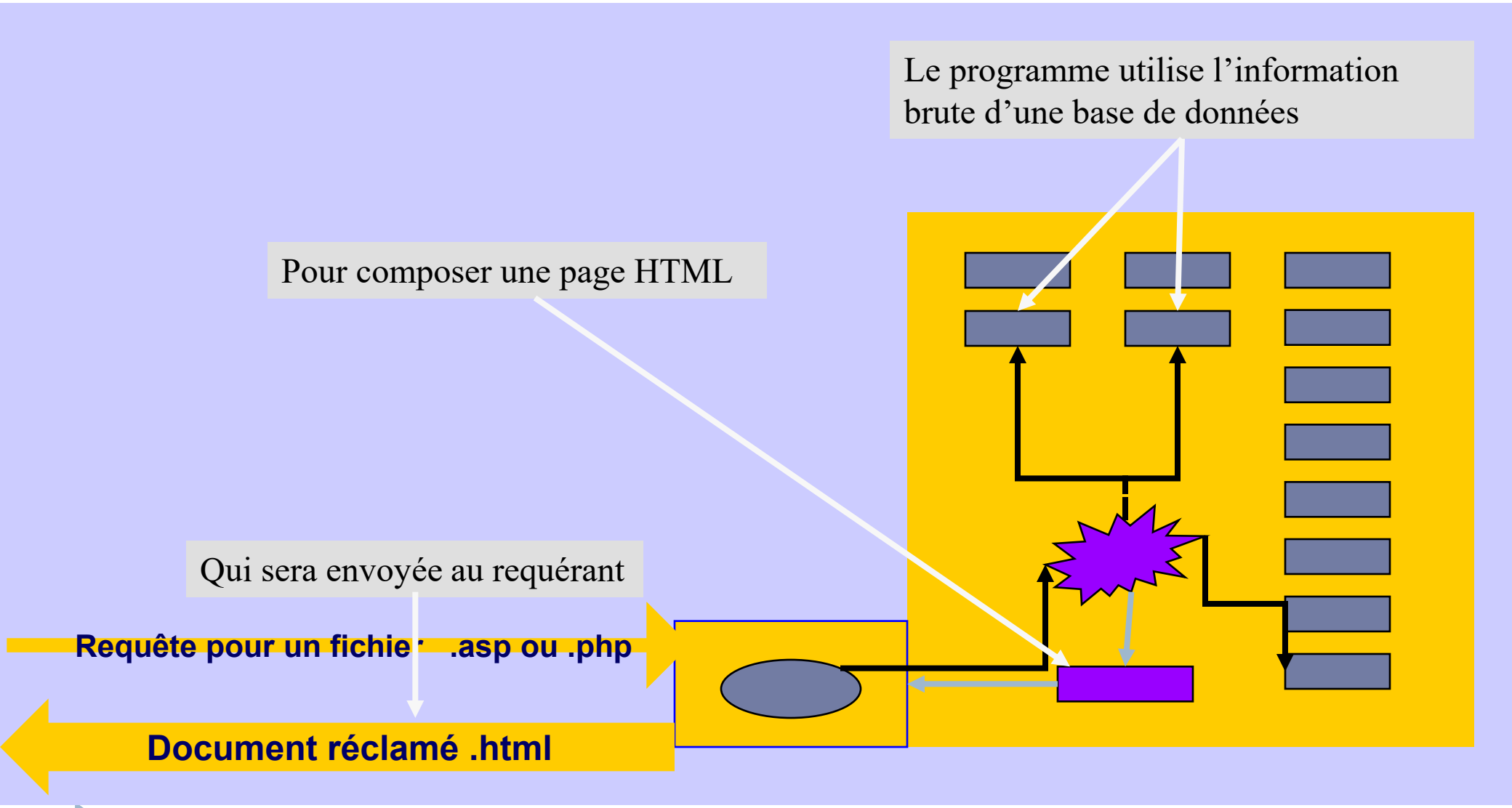
Introduction(3)



Introduction(4)

- Les langages de script-serveur : Définition
  - Un langage de script -serveur est :
    - un programme stocké sur un serveur et exécuté par celui-ci,
    - qui passe en revue les lignes d'un fichier source pour en modifier une partie du contenu,
    - avant de renvoyer à l'appelant (un navigateur par exemple) le résultat spécifique du traitement.
  - La tâche d'interprétation des ordres à exécuter est déléguée à un composant, souvent appelé moteur,
    - installé sur le serveur,
    - qui est doté d'une API et d'un fonctionnement identique quel que soit la plate-forme utilisée pour gérer le serveur

Introduction(5)



Introduction(6)

- Pages web dynamiques côté serveur ou côté client
  - **Langage côté client :** traité par la machine qui accueille le logiciel de navigation.
    - Ses résultats peuvent varier en fonction de plate-forme utilisée. Un programme en JavaScript pourra fonctionner sous Netscape et poser problème sous Internet explorer.
    - Les résultats peuvent être différents suivant la machine (PC, Mac)
    - Nécessité de tests importants
    - Ne permettent pas de masquer les sources du programme
    - Sont indépendants du serveur et donc de l'hébergement

Introduction(7)

- Pages web dynamiques côté serveur ou côté client
  - **Langage côté serveur** : le travail d'interprétation du programme est réalisé par le serveur
    - Sont indépendants de la machine et du logiciel de navigation utilisés pour la consultation.
    - Sont compatibles avec tous les navigateurs et toutes leurs versions.
    - Permettent de masquer les sources de ses programmes.
    - Nécessitent de recharger la page chaque fois que celle-ci est modifiée.
  - Pages côté serveur et côté client :
    - Script côté client pour des calculs et des traitement simples
    - Scripts côté serveur pour des calculs, des traitements et des mises à jours plus conséquents

Introduction(8)

- Quelques langages de création de pages web dynamiques **côté serveur**

Introduction(9)

- Les langages de création de page web dynamiques côté serveur
  - **PHP** (*Hypertext PreProcessor*)
    - Connaît un succès toujours croissant sur le Web et se positionne comme un grand rival d'ASP
    - L'environnement Linux est sa plateforme de prédilection
    - Combiné avec le serveur Web Apache et la base de données MySQL, PHP offre une solution particulièrement robuste, stable et efficace
    - Gratuité : Tous les logiciels sont issus du monde des logiciels libres (Open Source).

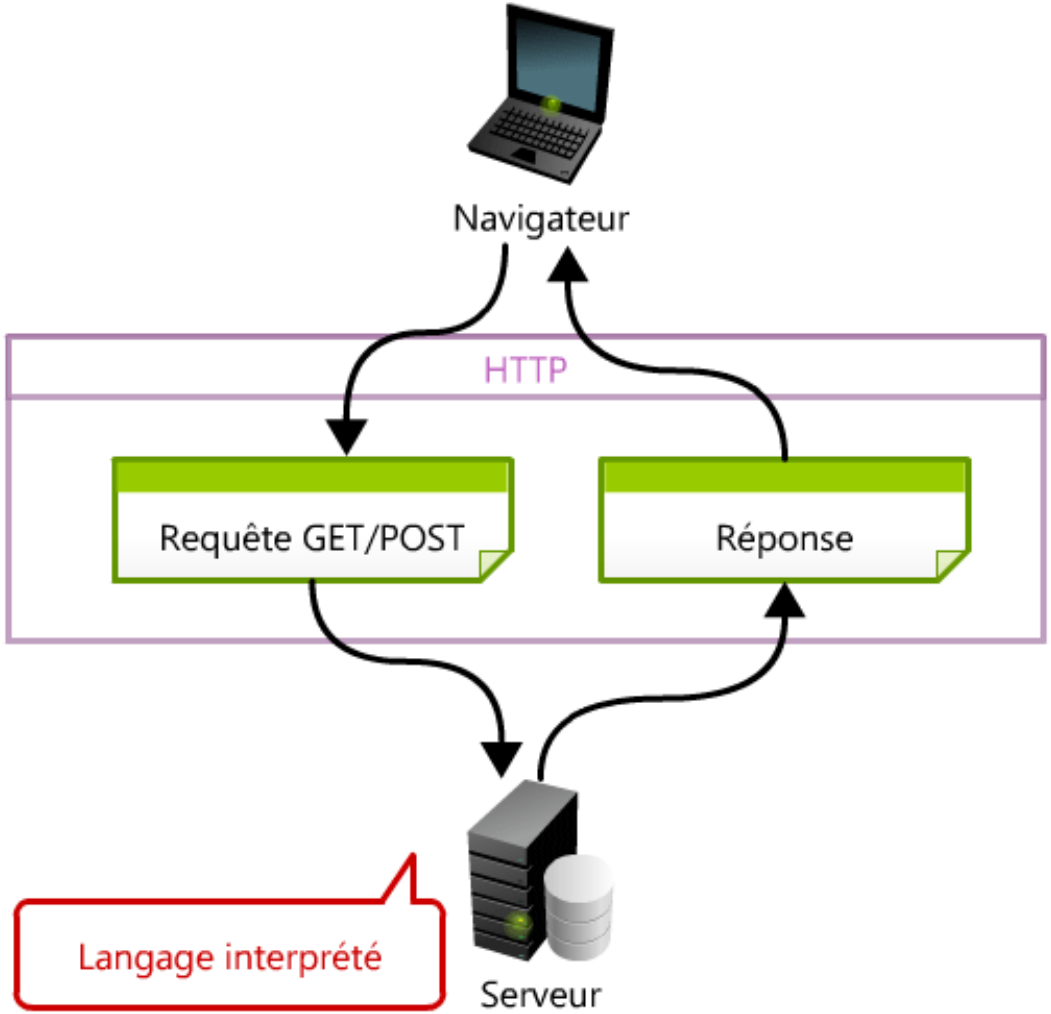
Histoire et Origine sur PHP

- La première version de PHP a été mis au point par Rasmus Lerdorf en 1994
  - Version appelée à l'époque Personal Home Pages
  - Pour conserver la trace des utilisateurs venant consulter son CV sur son site, grâce à l'accès à une base de données par l'intermédiaire de requêtes SQL
- La version 3.0 de PHP fut disponible le 6 juin 1998 devenu alors PHP: Hypertext Preprocessor
- A la fin de l'année 1999, une version bêta de PHP, baptisée PHP4 est apparue
- En 2001 cinq millions de domaines utilisent PHP
  - trois fois plus que l'année 2000
- En 2002, PHP est utilisé par plus de 8 millions de sites Web à travers le monde, en 2007 par plus de 20 millions et en 2013 par plus de 244 millions.
- La version 7, sortie le 3 décembre 2015. Elle utilise Zend Engine 2 et introduit une modélisation objet plus performante, une gestion des erreurs fondée sur le modèle des exceptions, ainsi que des fonctionnalités de gestion pour les entreprises.
- La version 8 qui est sortie en 2020 apporte plusieurs modifications et c'est la version en vigueur en 2022

C’est QUOI ?

- Définition
  - Un langage de scripts permettant la création d’applications Web
  - Indépendant de la plate-forme utilisée puisqu’il est exécuté côté serveur et non côté client.
  - La syntaxe du langage provient de celles du langage C, du Perl et de Java.
  - Ses principaux atouts sont:
    - La gratuité et la disponibilité du code source
    - La simplicité d'écriture de scripts
    - La possibilité d'inclure le script PHP au sein d'une page HTML
    - La simplicité d'interfaçage avec des bases de données
    - L'intégration au sein de nombreux serveurs web (Apache, Microsoft IIS, ...)

Fonctionnement de PHP



Fonctionnement de PHP

Il peut y avoir une étape supplémentaire, c'est-à-dire, celle du dialogue entre PHP et une base de données MySQL, PostgreSQL, etc.

Intégration PHP et HTML (1)

- Principe
  - Les scripts PHP sont généralement intégrés dans le code d'un document HTML
  - L'intégration nécessite l'utilisation de balises
    - avec le style xml : <? ligne de code PHP ?>
    - Avec le style php: <?php ligne de code PHP ?>
    - avec le style JavaScript :

      <script language="php"> ligne de code PHP </script>
    - avec le style des ASP : <% ligne de code ASP %>

Intégration PHP et HTML (2)

- Forme d'une page PHP
  - Intégration directe

```
< ?php
//ligne de code PHP
?>
<html>
<head> <title> Mon script PHP
</title> </head>
<body>
//ligne de code HTML
< ?php
//ligne de code PHP
?>
//ligne de code HTML
….
</body> </html>
```

Intégration PHP et HTML (3)

- Forme d'une page PHP
  - Inclure un fichier PHP dans un fichier HTML : include()

```
Fichier Prinipal

<html>
<head>
<title> Fichier d'appel </title>
</head>
<body>
<marquee> TP1 </marquee>
<br/>
<?php
echo("Code PHP");
include "information.php" ;
?>
</body>
</html>
```

```
Fichier à inclure : information.php


<?php
echo("je suis dans la page
information.php
\"afficher avec echo\"");
?>
<br/>
<?php
print(2*3);
print("\t \"afficher avec print\"");
?>
```

Enregistrer le fichier principal avec une extension .html et puis avec une extension .php et voir la différence!!!!

Intégration PHP et HTML (4)

- Envoi du code HTML par PHP
  - La fonction echo : echo Expression;
    - echo "Chaine de caracteres";
    - echo (1+2)*87;
  - La fonction print : print(expression);
    - print("Chaine de caracteres");
    - print ((1+2)*87);
  - La fonction printf : printf (chaîne formatée);
    - printf ("Le périmètre du cercle est %d",$Perimetre);

Intégration PHP et HTML (5)

- Exécuter un code PHP après validation d'un formulaire

Login.html

```
<html>
<body>
<marquee behavior="alternate"><h4><font color=blue><h2>Fichier .html appel un
fichier .php</h2></font></h4></marquee>
<br>
<br>
<br>
<table align=center>
<FORM   action="login.php"    method="GET">
<tr><td>Login</td><td><input type="text" name="log"></td></tr>
<tr><td>Passwd</td><td><input type="password" name="pwd"></td></tr>
 <tr><td><input type="submit" name="val" value="Valider"></td><td><input
type="reset" name="an" value="Effacer"></td></tr>
</FORM>
</table>
</body>
</html>
```

Intégration PHP et HTML (6)

- Exécuter un code PHP après validation d'un formulaire

Login.php

```
<?php
$lg=$_GET['log'];
$pd=$_GET['pwd'];
echo("Bonjour ".$lg);
?>
<br/>
<?php
echo("Ton mot de passe est: ".$pd);
?>
```

Syntaxe de base : Introduction

- Typologie
  - Toute instruction se termine par un point-virgule
  - Sensible à la casse
    - Sauf par rapport aux fonctions
- Les commentaires
  - /* Voici un commentaire! */
  - // un commentaire sur une ligne

Syntaxe de base : Les constantes

- Les constantes
  - **Define**("nom_constante", valeur_constante)
    - define (“ma_const”, “Vive PHP”) ;
    - define (“an”, 2018) ;
  - Les constantes prédéfinies
    - NULL
    - _FILE_
    - _LINE_
    - PHP_VERSION
    - PHP_OS
    - TRUE et FALSE
    - E_ERROR

Syntaxe de base : Les variables (1)

- Principe
  - Commencent par le caractère $
  - N'ont pas besoin d'être déclarées
- Fonctions de vérifications de variables
  - Doubleval(), empty(), gettype(), intval(),
  - is_array(), is_bool(), is_double(), is_float(), is_int(), is_integer, is_long(), is_object(), is_real(), is_numeric(), is_string()
  - Isset(), settype(), strval(), unset()
- Affectation par valeur et par référence
  - Affectation par valeur : $b=$a
  - Affectation par (référence) variable : $c = &$a

Syntaxe de base : Les variables(2)

- Visibilité des variables
  - Variable locale
    - Visible uniquement à l'intérieur d'un contexte d'utilisation
  - Variable globale
    - Visible dans tout le script
    - Utilisation de l'instruction global() dans des contextes locales

```
<?
$var = 100;
function test(){
global $var;
return $var;}
$resultat = test();
if ($resultat) echo $resultat; else echo " erreur ";
?>
```

Syntaxe de base : Les variables(3)

- Les variables dynamiques
  - Permettent d'affecter un nom différent à une autre variable

```
$nom_variable = 'nom_var';
$$nom_variable = valeur; // équivaut à $nom_var = valeur;
```

  - Les variables tableaux sont également capables de supporter les noms dynamiques

```
$nom_variable = array("val0", "val1", ...,"valN");
${$nom_variable[0]} = valeur; $val0 = valeur;
$nom_variable = "nom_var";
${$nom_variable}[0] = valeur;
$nom_var[0] = valeur;
```

  - Les accolades servent aussi à éviter toute confusion lors du rendu d'une variable dynamique

```
echo "Nom : $nom_variable - Valeur : ${$nom_variable}";
// équivaut à echo "Nom : $nom_variable - Valeur : $nom_var";
```

Syntaxe de base : Les variables (4)

- Variables prédéfinies
  - Les variables d'environnement dépendant du client

Variable	Description
$_SERVER["HTTP_HOST"]	Nom d'hôte de la machine du client (associée à l'adresse IP)
$_SERVER["HTTP_REFERER"]	URL de la page qui a appelé le script PHP
$_SERVER["HTTP_ACCEPT_LANGUAGE"]	Langue utilisée par le serveur (par défaut en-us)
$_SERVER["HTTP_ACCEPT"]	Types MIME (Multipurpose Internet Mail Extensions) reconnus par le serveur (séparés par des virgules)
$_SERVER["CONTENT_TYPE"]	Type de données présent dans le corps de la requête. Il s'agit du type MIME des données
$_SERVER["REMOTE_ADDR"]	L'adresse IP du client appelant le script CGI
$_SERVER["PHP_SELF"]	Nom du script PHP

Syntaxe de base : Les variables (5)

- Variables prédéfinies
  - Les variables d'environnement dépendant du serveur

Variable	Description
$_SERVER["SERVER_NAME"]	Le nom du serveur
$_SERVER["HTTP_HOST"]	Nom de domaine du serveur
$_SERVER["SERVER_ADDR"]	Adresse IP du serveur
$_SERVER["SERVER_PROTOCOL"]	Nom et version du protocole utilisé pour envoyer la requête au script PHP
$_SERVER["DATE_GMT"]	Date actuelle au format GMT
$_SERVER["DATE_LOCAL"]	Date actuelle au format local
$_SERVER["$DOCUMENT_ROOT"]	Racine des documents Web sur le serveur

Syntaxe de base : Les variables (6)

- Variables prédéfinies
  - Affichage des variables d'environnement
    - la fonction phpinfo()
      - <? phpinfo(); ?>
      - **echo phpinfo(constante);**

INFO_CONFIGURATION	affiche les informations de configuration.
INFO_CREDITS	affiche les informations sur les auteurs du module PHP
INFO_ENVIRONMENT	affiche les variables d'environnement.
INFO_GENERAL	affiche les informations sur la version de PHP.
INFO_LICENSE	affiche la licence GNU Public
INFO_MODULES	affiche les informations sur les modules associés à PHP
INFO_VARIABLES	affiche les variables PHP prédéfinies.

    - la fonction getenv()
      - <? echo getenv("HTTP_USER_AGENT");?>

Syntaxe de base : Les types de données

- Principe
  - Pas besoin d'affecter un type à une variable avant de l'utiliser
    - La même variable peut changer de type en cours de script
    - Les variables issues de l'envoi des données d'un formulaire sont du type string
- Les différents types de données
  - Les entiers : le type Integer
  - Les flottants : le type Double
  - Les tableaux : le type array
  - Les chaînes de caractères : le type string
  - Les objets

Syntaxe de base : Les types de données (2)

- Le transtypage
  - La fonction settype() permet de convertir le type auquel appartient une variable

```
<?  $nbre=10;
    Settype($nbre, " double ");
    Echo " la variable $nbre est de type " , gettype($nbre); ?>
```

  - Transtypage explicite : le cast
    - (int), (integer) ; (real), (double), (float); (string); (array); (object)

```
<? $var=" 100 FRF ";
    Echo " pour commencer, le type de la variable est $var, gettype($var);
    $var =(double) $var;
    Echo <br> Après le cast, le type de la variable est $var ", gettype($var);
    Echo "<br> et a la valeur $var "; ?>
```

- Détermination du type de données
  - Gettype(), Is_long(), Is_double(), Is_string(), Is_array(), Is_object(), Is_bool()

Syntaxe de base : Les chaînes de caractères(1)

- Principe
  - Peuvent être constituées de n'importe quel caractère alphanumérique et de ponctuation, y compris les caractères spéciaux

```
\tLa nouvelle monnaie unique, l' €uro, est enfin là...\n\r
```

  - Une chaîne de caractères doit être toujours entourée par des guillemets simples (')ou doubles (")

" Ceci est une chaîne de caractères valide."
'Ceci est une chaîne de caractères valide.'
"Ceci est une chaîne de caractères invalide.'

  - Des caractères spéciaux à insérer directement dans le texte, permettent de créer directement certains effets comme des césures de lignes

Car	Code ASCII	Code hex	Description
\car			échappe un caractère spécifique.
" "	32	0x20	un espace simple.
\t	9	0x09	tabulation horizontale
\n	13	0x0D	nouvelle ligne
\r	10	0x0A	retour chariot
\0	0	0x00	caractère NUL
\v	11	0x0B	tabulation verticale

Syntaxe de base : Les chaînes de caractères(2)

Quelques fonctions de manipulation

chaîne_result = addCSlashes(chaîne, liste_caractères);
ajoute des slashs dans une chaîne

chaîne_result = addSlashes(chaîne);
ajoute un slash devant tous les caractères spéciaux.

chaîne_result = chop(chaîne);
supprime les espaces blancs en fin de chaîne.

caractère = chr(nombre);
retourne un caractère en mode ASCII

chaîne_result = crypt(chaîne [, chaîne_code])
code une chaîne avec une base de codage.

echo expression_chaîne;
affiche à l'écran une ou plusieurs chaînes de caractères.

$tableau = explode(délimiteur, chaîne);
scinde une chaîne en fragments à l'aide d'un délimiteur et retourne un tableau.

Syntaxe de base : les opérateurs (1)

- Les opérateurs
  - les opérateurs de calcul
  - les opérateurs d'assignation
  - les opérateurs d'incrémentation
  - les opérateurs de comparaison
  - les opérateurs logiques
  - les opérateurs bit-à-dit
  - les opérateurs de rotation de bit

Syntaxe de base : Les opérateurs(2)

- Les opérateurs de calcul

Opérateur	Dénomination	Effet	Exemple	Résultat si x=7
+	opérateur d'addition	Ajoute deux valeurs	$x+3	10
-	opérateur de soustraction	Soustrait deux valeurs	$x-3	4
*	opérateur de multiplication	Multiplie deux valeurs	$x*3	21
/	plus: opérateur de division	Divise deux valeurs	$x/3	2.3333333
=	opérateur d'affectation	Affecte une valeur à une variable	$x=3	Met la valeur 3 dans la variable $x

Syntaxe de base : Les opérateurs(3)

- Les opérateurs d'assignation

Opérateur	Effet
+=	addition deux valeurs et stocke le résultat dans la variable (à gauche)
-=	soustrait deux valeurs et stocke le résultat dans la variable
*=	multiplie deux valeurs et stocke le résultat dans la variable
/=	divise deux valeurs et stocke le résultat dans la variable
% =	donne le reste de la division deux valeurs et stocke le résultat dans la variable
| =	Effectue un OU logique entre deux valeurs et stocke le résultat dans la variable
^ =	Effectue un OU exclusif entre deux valeurs et stocke le résultat dans la variable
& =	Effectue un Et logique entre deux valeurs et stocke le résultat dans la variable
.=	Concatène deux chaînes et stocke le résultat dans la variable

Syntaxe de base : Les opérateurs(4)

- Les opérateurs d'incrémentation

Opérateur	Dénomination	Effet	Syntaxe	Résultat (avec x valant 7)
++	Incrémentation	Augmente d'une unité la variable	$x++	8
--	Décrémentation	Diminue d'une unité la variable	$x--	6

- Les opérateurs de comparaison

Opérateur	Dénomination	Effet	Exemple	Résultat
==	opérateur d'égalité	Compare deux valeurs et vérifie leur égalité	$x==3	Retourne 1 si $X est égal à 3, sinon 0
<	opérateur d'infériorité stricte	Vérifie qu'une variable est strictement inférieure à une valeur	$x<3	Retourne 1 si $X est inférieur à 3, sinon 0
<=	opérateur d'infériorité	Vérifie qu'une variable est inférieure ou égale à une valeur	$x<=3	Retourne 1 si $X est inférieur à 3, sinon 0
>	opérateur de supériorité stricte	Vérifie qu'une variable est strictement supérieure à une valeur	$x>3	Retourne 1 si $X est supérieur à 3, sinon 0
>=	opérateur de supériorité	Vérifie qu'une variable est supérieure ou égale à une valeur	$x>=3	Retourne 1 si $X est supérieur ou égal à 3, sinon 0
!=	opérateur de différence	Vérifie qu'une variable est différente d'une valeur	$x!=3	Retourne 1 si $X est différent de 3, sinon 0

Syntaxe de base : Les opérateurs(5)

Les opérateurs logiques

Opérateur	Dénomination	Effet	Syntaxe
|| ou **OR**	OU logique	Vérifie qu'une des conditions est réalisée	((condition1)||(condition2))
&& ou **AND**	ET logique	Vérifie que toutes les conditions sont réalisées	((condition1)&&(condition2))
XOR	OU exclusif	Opposé du OU logique	((condition1)XOR(condition2))
!	NON logique	Inverse l'état d'une variable booléenne (retourne la valeur 1 si la variable vaut 0, 0 si elle vaut 1)	(!condition)

Les opérateurs bit-à-bit

Opérateur	Dénomination	Effet	Syntaxe	Résultat
&	ET bit-à-bit	Retourne 1 si les deux bits de même poids sont à 1	9 & 12 (1001 & 1100)	8 (1000)
|	OU bit-à-bit	Retourne 1 si l'un ou l'autre des deux bits de même poids est à 1 (ou les deux)	9 | 12 (1001 | 1100)	13 (1101)
^	OU bit-à-bit	Retourne 1 si l'un des deux bits de même poids est à 1 (mais pas les deux)	9 ^ 12 (1001 ^ 1100)	5 (0101)
~	Complément (NON)	Retourne 1 si le bit est à 0 (et inversement)	~9 (~1001)	6 (0110)

Syntaxe de base : Les opérateurs(6)

- Les opérateurs de rotation de bit

Opérateur	Dénomination	Effet	Syntaxe	Résultat
<<	Rotation à gauche	Décale les bits vers la gauche (multiplie par 2 à chaque décalage). Les zéros qui sortent à gauche sont perdus, tandis que des zéros sont insérés à droite	6 << 1 (110 << 1)	12 (1100)
>>	Rotation à droite avec conservation du signe	Décale les bits vers la droite (divise par 2 à chaque décalage). Les zéros qui sortent à droite sont perdus, tandis que le bit non-nul de poids plus fort est recopié à gauche	6 >> 1 (0110 >> 1)	3 (0011)

- Autres opérateurs

Opérateur	Dénomination	Effet	Syntaxe	Résultat
.	Concaténation	Joint deux chaînes bout à bout	"Bonjour"."Au revoir"	"BonjourAu revoir"
$	Référencement de variable	Permet de définir une variable	$MaVariable = 2;	
->	Propriété d'un objet	Permet d'accéder aux données membres d'une classe	$MonObjet->Propriete	

Syntaxe de base : Les opérateurs(7)

- Les priorités

Priorité des opérateurs												
	()	[]										
	--	++	!	~	-							
	*	/	%									
	+	-										
	<	<=	>=	>								
	==	!=										
	&											
	^											
	|											
	&&											
	||											
	?	:										
	=	+=	-=	*=	/=	%=	<<=	>>=	>>>=	&=	^=	|=
	AND											
	XOR											

Syntaxe de base : Les instructions conditionnelles(1)

- L'instruction if
  - if (condition réalisée) { liste d'instructions }
- L'instruction if ... Else
  - if (condition réalisée) {liste d'instructions} else { autre série d'instructions }
- L'instruction if ... elseif ... Else
  - if (condition réalisée) {liste d'instructions} elseif (autre condition) {autre série d'instructions } else (dernière condition réalisée) { série d'instructions }
- Opérateur ternaire
  - (condition) ? instruction si vrai : instruction si faux

Syntaxe de base : Les instructions conditionnelles(2)

- L'instruction switch

```
switch (Variable) {
case Valeur1: Liste d'instructions break;
case Valeur1: Liste d'instructions break;
case Valeurs...: Liste d'instructions break;
default: Liste d'instructions break;
}
```

Syntaxe de base : Les instructions conditionnelles(3)

- La boucle for
  - for ($i=1; $i<6; $i++) { echo "$i
"; }
- La boucle while
  - While(condition) {bloc d'instructions ;}
  - While (condition) :Instruction1 ;Instruction2 ;
    …. endwhile ;
- La boucle do…while
  - Do {bloc d'instructions ;}while(condition) ;
- La boucle foreach
  - Foreach ($tableau as $valeur) {insts utilisant $valeur ;}

PARTIE II :

L'INTERACTIVITÉ EN PHP

PHP et les formulaires(1)

Formulaire HTML

- Retourne des informations saisies par un utilisateur vers une application serveur
- La création d'un formulaire nécessite la connaissance de quelques balises HTML indispensables, CSS et JavaScript:

  - Structure : un formulaire commence toujours par la balise
    <form> et se termine par la balise </form>

  - Champ de saisie de text en ligne :
    <input type = "text" name ="nom_du_champ" value="chaîne">

  - Boutons d'envoi et d'effacement :
    <input type=" submit " value = "Envoyer">
    <input type = "reset" name ="efface" value = "Effacer">

  - Case à cocher et bouton radio :
    <input type = "checkbox" name ="case1" value="valeur_case">
    <input type = "radio" name ="radio1" value ="valeur_radio">

PHP et les formulaires(2)

- Liste de sélection avec options à choix unique :

```
<select name ="select" size="1">
<option value = "un"> choix </option>
<option value ="deux"> choix2 </option>
</select>
```

- Liste de sélection avec options à choix multiples :

```
<select name ="select" size = "1" multiple>
<option value = "un"> choix1 </option>
<option value = "deux"> choix2 </option>
</select>
```

PHP et les formulaires(3)

- Méthodes d'envoi get et post
  - transmission selon une des deux méthodes d'envoi GET ou POST
    - **La méthode GET** place les informations d'un formulaire directement à la suite de l'adresse URL de la page appelée.
      - http://www.site.com/cible.php?champ=valeur&champ2=valeur
      - inconvénients :
        - rendre visibles les données dans la barre d'adresse du navigateur.
        - De plus, la longueur totale est limitée à 255 caractères, ce qui rend impossible la transmission d'un volume de données important
    - **La méthode POST** regroupe les informations dans l'entête d'une requête HTTP
      - Assure une confidentialité efficace des données

PHP et les formulaires(4)

- Récupération des paramètres en PHP
  - Si la directive register_globals de php.ini est TRUE, lors de la soumission (submit), chaque élément de saisie est assimilé à une variable PHP dont le nom est constitué par la valeur de l'attribut name et son contenu par la valeur de l'attribut value
  - Les tableaux associatifs $_GET et $_POST contiennent toutes les variables envoyées par un formulaire

PHP et les formulaires(5)

```
<html>
<body>
<form method="GET | POST" action="Tp_interaction1.php">
<input type="text" name="Champ_saisie"/>
<br/>
<select name="Liste_Choix" size="3">
<option value="Option_1">Option_1</option>
<option value="Option_2">Option_2</option>
<option value="Option_3">Option_3</option>
</select>
<br/>
<textarea name="Zone_Texte" cols="30" rows="5"> Texte par défaut  </textarea>
<br/>
    <input type="checkbox" name="Case_Cocher[]" value="Case_1"> Case 1<br>
    <input type="checkbox" name="Case_Cocher[]" value="Case_2"> Case 2<br>
    <input type="checkbox" name="Case_Cocher[]" value="Case_3"> Case 3<br>
    <input type="radio" name="Case_Radio" value="Case radio 1"> radio 1<br>
    <input type="radio" name="Case_Radio" value="Case radio 2"> radio 2<br>
    <input type="radio" name="Case_Radio" value="Case radio 3"> radio 3<br>
    <input type="reset" name="Annulation" value="Annuler">
 <input type="submit" name="Soumission" value="Soumettre">
</form>
</body>
</html>
```

PHP et les formulaires(6)

Tp_interaction1.php

```
<?php
  $resultat = $_GET["Champ_saisie"] . "<br>";
  $resultat .= $_GET["Liste_Choix"] . "<br>";
  $resultat .= $_GET["Zone_Texte"] . "<br>";
  for ($i = 0; $i < count($_GET["Case_Cocher"]); $i++)
  {
    $resultat .= $_GET["Case_Cocher"][$i] . "<br>";
  }
  $resultat .= $_GET["Case_Radio"] . "<br>";
  echo $resultat;
  ?>
```

PHP et les formulaires(7)

- PHP et les formulaires
  - La plupart des éléments d'un formulaire n'acceptent qu'une seule et unique valeur, laquelle est affectée à la variable correspondante dans le script de traitement.

```
$Champ_Saisie ← "Ceci est une chaîne de caractères.";
```

  - Pour des cases à cocher et les listes à choix multiples, plusieurs valeurs peuvent être sélectionnées entraînant l'affectation d'un tableau de valeurs aux variables correspondantes.

```
$Case_Cocher[0] ← "Case 1";
$Case_Cocher[2] ← "Case  3";
```

PHP et les formulaires(8)

- Variables **superglobales**
  - Nous avons déjà vu les tableaux $_GET et $_POST qui sont des **variables superglobales.** Leur nom est écrit en majuscules et commence par un *underscore*, mais surtout ces variables sont générées automatiquement par PHP.
  - $_GET et $_POST sont plutôt faits pour transmettre les informations une seule fois, d'une page à une autre. On sait ainsi envoyer d'une page à une autre le nom et le prénom du visiteur, mais dès qu'on charge une autre page ces informations sont « oubliées ».
  - Il existe d'autres types de variables **superglobales**.
  - Parmi elles, certaines permettent de stocker des informations pendant la durée d'une visite, **c'est le principe des variables sessions,** mais aussi de stocker des informations sur l'ordinateur de vos visiteurs pendant plusieurs mois, **c'est le principe des cookies.**

Les sessions(1)

- Principe
  - Est un mécanisme permettant de **mettre en relation** les différentes requêtes du même client sur une période de temps donnée.
  - Les sessions permettent de conserver des informations relatives à un utilisateur lors de son parcours sur un site web
  - Des données spécifiques à un visiteur pourront être transmises de page en page afin d'adapter personnellement les réponses d'une application PHP
  - Chaque visiteur en se connectant à un site reçoit un numéro d'identification dénommé identifiant de session (SID)
  - La fonction **session_start()** se charge de générer automatiquement cet identifiant unique de session et de créer un répertoire. Elle **doit être placée au début** de chaque page afin de démarrer ou de continuer une session.

```
<?php
 session_start();
 $Session_ID = session_id();
 // $Session_ID = 7edf48ca359ee24dbc5b3f6ed2557e90    ?>
```

Les sessions(2)

- Principe
  - Un répertoire est créé sur le serveur à l'emplacement désigné par le fichier de configuration php.ini, afin de recueillir les données de la nouvelle session.

```
[Session]
session.save_path= C:\PHP\sessiondata
; Rép session = \sess_7edf48ca359ee24dbc5b3f6ed2557e90
```

  - Le fichier php.ini peut également préciser un nom de session par l'option session.name ou sa durée de vie par session.gc_maxlifetime
  - La session en cours peut être détruite par la fonction session_destroy(). Cette commande supprime toutes les informations relatives à l'utilisateur.

```
session_destroy();
```

Les sessions(3)

- Le traitement des variables de session
  - Les variables de session sont chargées dans une session par l'intermédiaire de la fonction session_register()

```
<?php
 session_start();
 session_register("nom_variable");
 ...
 session_register("nom_variableN");
?>
```

  - Une fois la variable enregistrée, elle est accessible à travers le tableau associatif $_SESSION["nom_variable"]

Les sessions(4)

- Le traitement des variables de session

Le transport des informations entre les documents est réalisé par l'entremise

- soit d'un cookie
- soit d'une requête HTTP
  - Cette dernière solution est la plus fiable puisque les cookies peuvent ne pas être acceptés par le client ou celui-ci pourrait les détruire en cours de session.
  - Il suffit de concaténer l'identifiant de session à l'adresse URL de la page cible pour que cette dernière puisse accéder aux informations conservées par la session.

```
echo '<a href=" http://www.site.com/doc.php? '. session_name(). '=' . session_id().'> lien</a>'
```

Les sessions(5)

- Le traitement des variables de session

  - Par défaut, PHP tente de passer par les cookies pour sauvegarder l'identifiant de session dans le cas où le client les accepterait. Il est possible d'éviter cela, il suffit de désactiver l'option de configuration session.use_cookies dans le fichier php.ini.

```
[Session]
session.use_cookies 0; //désactive la gestion des sessions par cookie
```

Les sessions(6)

Exemple

```
<!-- Fichier : formulaire.html -->
<html><body>
   <form method="POST" action="traitement.php">
    <table border="0">
      <tr>
        <td><u>Nom :</u></td>
        <td><input type="text" name="Nom" size="20" value="FAYE"></td></tr>
      <tr>
        <td><u>Prénom :</u></td>
        <td><input type="text" name="Prenom" size="20" value="Jean-Pierre"></td></tr>
      <tr>
        <td><u>eMail :</u></td>
        <td><input type="text" name="cEmail" size="20" value="du@du.com"></td></tr>
      <tr><td> </td>
        <td><input type="submit" name="soumettre" value="Envoyer"></td></tr></table>
</form></body ></html>
```

Les sessions(7)

traitement.php

```
<?php
 session_start();
 $nom = $_POST["Nom"];
 $prenom = $_POST["Prenom"];
 $email = $_POST["cEmail"];
//il ne fonctionnera pas dans les environnements où register_globals est désactivée dans php.ini
 session_register("nom");
 session_register("prenom");
 session_register("email");
 $_SESSION["nom"]=$nom;
 $_SESSION["prenom"]=$prenom;
 $_SESSION["email"]=$email;
//redirection avec la fonction header
 header("Location: session.php?" . session_name() . "=" . session_id());
?>
```

Les sessions(8)

session.php

```
<?php
    session_start();
?>
<html><body><?php
echo("<u>Identifiant de session :</u> <b>"
                        . session_id() . "</b><br>");
    echo("<u>Nom de la session :</u> <b>"
                        . session_name() . "</b><br><br>");
    echo("<u>Nom :</u> <b>". $_SESSION["nom"] . "</b><br>");
    echo("<u>Prénom :</u> <b>" . $_SESSION["prenom"] . "</b><br>");
    echo("<u>eMail :</u> <b>" . $_SESSION["email"] . "</b><br>");
    //session_destroy();
  ?>
 </body>
</html>
```

NB : il n'y a pas de balise <form … >

Les sessions(9)

Exemple2.php

```
<?php
// On démarre la session AVANT d'écrire du code HTML
session_start();
// On va créer quelques variables de session dans $_SESSION
$_SESSION['prenom'] = 'Pascal';
$_SESSION['nom'] = 'FAYE';
$_SESSION['age'] = 10;
?>
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<title>Titre de ma page</title>
</head>
<body>
<p>
Salut <?php echo $_SESSION['prenom']; ?> !<br />
Tu es à l'accueil de mon site (index.php). Tu veux aller sur une autre page ?
</p>
<p>
<a href="informations.php">Lien vers informations.php</a><br />
<a href="mapage.php">Lien vers mapage.php</a><br />
<a href="monscript.php">Lien vers monscript.php</a>
</p>
</body>
</html>
```

Les sessions(10)

informations.php

```
<?php
session_start(); // On démarre la session AVANT toute chose
?>
<!DOCTYPE html>
<html>
<head>
<meta charset="utf-8" />
<title>Titre de ma page</title>
</head>
<body>
<p>Re-bonjour !</p>
<p>
Je me souviens de toi ! Tu t'appelles <?php echo $_SESSION['prenom'] . ' ' . $_SESSION['nom']; ?> !<br />
Et tu as <?php echo $_SESSION['age']; ?> ans, c'est ça ?
</p>
</body>
</html>
```

Les sessions (11)

- Les fonctions de sessions

session_start() -- Initialise les données de session

session_id() -- Affecte et/ou retourne l'identifiant de session courante

session_name() -- Affecte et/ou retourne le nom de la session courante

session_register() -- Enregistre une variable dans la session courante

session_destroy() -- Détruit toutes les données enregistrées d'une session

session_is_registered() -- Indique si une variable a été enregistrée dans la session ou pas

session_unregister() -- Supprime une variable dans la session courante

session_unset() -- Détruit toutes les variables de session

session_cache_expire() -- Retourne la date d'expiration du cache de la session

session_save_path() -- Affecte et/ou retourne le chemin de sauvegarde de la session courante

session_decode() -- Décode les données de session à partir d'une chaîne

session_encode() -- Encode les données de session dans une chaîne

Les sessions (12)

- Quelles sont les erreurs possibles ?
  - **Répertoire de session inaccessible**

Warning: open(/tmp\sess_3c80883ca4e755aa72803b05bce40c12, O_RDWR) failed: m (2) in c:\phpdev\www\bp\header.php on line 2

Le répertoire de sauvegarde est défini dans le php.ini : `session.save_path = /tmp`

Il faut donc

Créer un répertoire

Lui donner les droits d'écriture pour tous

En spécifier le chemin dans le **php.ini**

  - **Par défaut PHP n'est pas autorisé à utiliser les sessions**

Il faut s'assurer que le PHP est bien autorisé a créer des sessions. C'est juste un paramètre à activer. Faire un phpinfo() pour voir ces paramètres. La commande phpinfo() se contente d'afficher dans le navigateur le contenu du fichier de configuration php.ini.

Les sessions (13)

- Quelles sont les erreurs possibles ?
  - **Avoir déjà écrit dans la page**

Warning: Cannot send session cookie - headers already sent by (output started at /home/SiteWeb/SiteAnalyse/index.php:3) in /home/SiteWeb/SiteAnalyse/index.php on line 6

Cette erreur survient lorsqu'on tente d'ouvrir une session après avoir déjà écrit dans le document, ce qui interdit.

Ce qu'il ne faut pas faire :

```
<html>
<body>
<?php session_start();
...
```

ceci non plus :

```
<?php echo "<html>";
...
session_start();
```

Les cookies (1)

- Principe
  - Un cookie est un fichier texte créé par un script que le serveur demande au client d'écrire sur son disque dur et qui permettent à un script de sauvegarder des variables liées à l'utilisateur.
  - Les cookies permettent de conserver des renseignements utiles sur chaque utilisateur, et de les réutiliser lors de sa prochaine visite
    - Exemple : personnaliser la page d'accueil ou les autres pages du site
      - un message personnel comportant par exemple son nom, la date de sa dernière visite, ou tout autre particularité.
  - Les cookies étant stockés sur le poste client, l'identification est immédiate et ne concernent que les renseignements qui le concernent
  - Pour des raisons de sécurité, les cookies ne peuvent être lus que par des pages issues du serveur qui les a créés

Les cookies(2)

- Principe
  - Le nombre de cookies qui peuvent être définis sur le même poste client est limité à 20 et la taille de chacun est limitée à 4ko.
  - Un navigateur peut stocker un maximum de 300 cookies.
  - La date d'expiration des cookies est définie de manière explicite par le serveur web chargé de les mettre en place.
  - Les cookies disponibles sont importés par PHP sous forme de variables identifiées sous les noms utilisés par ces cookies.
  - La variable globale du serveur $_COOKIES enregistre tous les cookies qui ont été définis.

Les cookies(3)

- Exemple d'application des cookies
  - Mémorisation des paniers dans les applications d'e-commerce
  - Identification des utilisateurs
  - Des pages web individualisées
    - Afficher des menus personnalisés
    - Afficher des pages adaptées aux utilisateurs en fonction de leurs visites précédentes.
- Mais attention, plusieurs risques sont liés à l'utilisation des cookies:
  - Le client peut refuser les cookies. Alors chaque visite vous lui direz "Bonjour, c'est la première fois que vous visitez ce site"...
  - Le client peut avoir effacé les cookies et alors d'un jour à l'autre vous avez perdu les informations
  - Le client peut changer de navigateur ou d'ordinateur d'une visite à l'autre
  - Etc.

Les cookies(4)

- Écrire des cookies
  - L'écriture de cookies est possible grâce à la fonction setcookie()
    - il faut cette fonction dès le début du script avant l'envoi d'aucune autre information de la part du serveur vers le poste client.

```
Setcookie("nom_var",  "valeur_var",  date_expiration,  "chemin",
  "domain", " secure")
```

```
<?php
setcookie ("PremierCookie", "Salut", time() +3600*24*7) ;
…
$Cookie1=$_COOKIE[" PremierCookie "]
if (!$Cookie1) {
  echo "le cookie n'a pas été défini";}
else {
echo $Cookie1."<br>";
}
?>
```

Les cookies(5)

- Écriture de cookies
  - **Nom_var :** nom de la variable qui va stocker l'information sur le poste client et qui sera utilisée pour récupérer cette information dans la page qui lira le cookie.
    - C'est la seule indication obligatoire pour un cookie
  - **Valeur_var :** valeur stockée dans la variable. Par exemple une chaîne de caractères ou une variable chaîne, en provenance d'un formulaire
  - **Date_expiration :** la date à laquelle le cookie ne sera plus lisible et sera effacé du poste client
    - on utilise en général la date du jour, définie avec la fonction time() à laquelle on ajoute la durée de validité désirée
    - Si l'attribut n'est pas spécifié, le cookie expire à l'issue de la session

Les cookies(6)

- Écriture de cookies
  - **Chemin :** définit la destination (incluant les sous-répertoire) à laquelle le navigateur doit envoyer le cookie.
  - **Domain set :** le nom du domaine à partir duquel peuvent être lus les cookies.
    - On peut aussi utiliser la variable d'environnement $SERVER_NAME à la place.
  - **Secure :** un nombre qui vaut 0 si la connexion n'est pas sécurisée, sinon, il vaut 1 pour une connexion sécurisée

Les cookies(7)

- Lecture de cookies
  - Si la directive register_globals de php.ini est TRUE, un cookie sera automatiquement passé en paramètre au script et sa valeur sera directement accessible dans la variable $NomDuCookie.
  - Les cookies sont accessibles dans le tableau associatif $_COOKIE
    - Si celui-ci visite une des pages PHP de ce même domaine dont le chemin est inclut dans le paramètre chemin (si le chemin est / le cookie est valide pour toutes les pages de ce site).
    - Il faut d'abord vérifier l'existence des variables dont les noms et les valeurs ont été définis lors de la création du cookie.
      - Cette vérification s'effectue grâce à la fonction isset($_COOKIE["nom_var"]) qui renvoie true si la variable $nom_var existe et false sinon.

```
<?php
if (isset($_COOKIE["nom_var"])){
echo "<h2> Bonjour isset($_COOKIE["nom_var"]</h2>" ;}
else echo "pas de cookie" ;
?>
```

Les cookies(8)

- Écriture de plusieurs variables par un cookie
  - Utilisation de la fonction compact() pour transformer les variables en un tableau
  - Convertir le tableau en une chaîne de caractère à l'aide de la fonction implode()
  - Affecter la chaîne créée à l'attribut « nom_cookie »

```
<?
$col="#FF0000" ;
$size=24;
$font="Arial" ;
$text="Je suis le ookie« ;
$arr=compact("col" , " size" , " font" , "text" );
$val=implode(" &" , $arr);
Setcookie("la_cookie" , $val, time()+600);
```

Les cookies(9)

- Lecture de plusieurs variables d'un cookie
  - Décomposé la chaîne stockée par la cookie et retrouver un tableau en utilisant la fonction explode()

```
<?php
echo " <b> voici le contenu de la chaîne cookie : </b><br>";
echo $le_cookie, "<br> <br>";
}
$arr=explode("&", $la_cookie);
echo "<b> ces variables ont été établies à partir de la chaîne
  cookie : </b> <br> <br>";
foreach ($arr as $k=>$elem) {
echo "$k=>$elem <br>";
}
?>
```

Les cookies(10)

- Supprimer un cookie
  - Il suffit de renvoyer le cookie grâce à la fonction setcookie() en spécifiant simplement l'argument NomDuCookie

```
<?php
setcookie("Visites");
?>
```

  - Une autre méthode consiste à envoyer un cookie dont la date d'expiration est passée:

```
<?php
setcookie("Visites","",time()-1 )
?>
```

Les cookies(11)

- Quelques précisions sur les cookies
  - Setcookie() doit être utilisée avant l'envoi de données HTML vers le navigateur
  - Le cookie n'est pas visible avant le prochain chargement de page
  - Avec PHP3, si plusieurs cookies sont envoyées de suite, les appels seront traités en ordre inverse, alors qu'avec PHP 4 ou 5 il seront traités dans l'ordre séquentiel.
  - Certains navigateurs ne traitent pas bien certains cas liés aux cookies
    - Microsoft Internet Explorer 4 avec le Service Pack 1 ne traite pas correctement les cookies qui ont le paramètre chemin défini
    - Netscape Communicator 4.05 et Microsoft Internet Explorer 3.x ne traitent pas correctement les cookies qui n'ont pas les paramètres chemin et expiration définis.

Les cookies(12)

- Exemples d'utilisation de cookies

```
//Un script permettant de savoir si un visiteur est déjà venu sur le site
  pendant le mois
setcookie("Visites","Oui",time()+2592000,"/", " .mondomaine.fr",0);

// Envoi d'un cookie qui disparaitra après la fermeture du navigateur
SetCookie("UserSessionCookie",$login.”:”.$pass);

// Envoi d'un cookie qui restera présent 24 heures
SetCookie("DejaVisite","1",time()+3600*24,"/",". mondomaine.fr ",0);

// Envoi d'un cookie qui s'effacera le 1er janvier 2003
SetCookie("An2002","1",mktime(0,0,0,1,1,2003),"/",". mondomaine.fr ",0);

//script permettant de compter le nombre de visite de la page par le
  visiteur
  <?php
  $Visites++;
  setcookie("Visites",$Visites,time()+2592000,"/",". mondomaine.fr ",0);?>
```

Interfaçage avec une base de données(1)

- Principe
  - PHP propose de nombreux outils permettant de travailler avec la plupart des SGBDR
    - Oracle, Sybase, Microsoft SQL Server, PostgreSQL ou encore MySQL
  - Lorsqu'une base de données n'est pas directement supportée par Php, il est possible d'utiliser un driver ODBC (pilote standard) pour communiquer avec les bases de données
  - Php fournit un grand choix de fonctions permettant de manipuler les bases de données.
    - Quatre fonctions sont essentielles:
      - La fonction de connexion au serveur
      - La fonction de choix de la base de données
      - La fonction de requête
      - La fonction de déconnexion

Interfaçage avec une base de données(2)

- Principe
  - Avec le SGBD MySQL, les fonctions de manipulation sont :
    - mysqli_connect
    - mysqli_select_db
    - mysqli_query
    - mysqli_close

TP
PHP-BD